

Cloud DW Assignment

1. AWS Redshift Assignment

Create an AWS Redshift Serverless with the ticketdb sample database.
Compare the costs of the following two queries:

Query 1

```
select sum(qtysold)
from sales
where listid in (select listid from listing where listtime > '2008-12-26');
```

Query 2

```
select sum(qtysold)
from sales
where saletime > '2008-12-26'
and listid in (select listid from listing where listtime > '2008-12-26');
```

Submit

(1) Screenshot of queryplan for each of the two queries above.

The screenshot shows the AWS Redshift console interface. At the top, there are tabs for 'Untitled 1', 'Untitled 2', and 'Untitled 4'. Below the tabs, there are buttons for 'Run', 'Limit 100', 'Explain', 'Isolated session', and a dropdown menu for 'Serverless: de...'. There is also a dropdown menu for 'sample_ticket...'. Below these buttons, there are icons for 'Schedule', 'Save', 'Fullscreen', and 'More'. The main area displays a SQL query:

```
1 explain
2 select sum(qtysold)
3 from sales
4 where listid in (select listid from listing where listtime > '2008-12-26');
5
6
```

Below the query, there are tabs for 'Result 1 (7)' and 'Result 2 (8)'. To the right of these tabs are buttons for 'Export', 'Chart', and a dropdown menu. The 'Result 1 (7)' tab is selected, showing a query plan:

QUERY PLAN
XN Aggregate (cost=4597.06..4597.06 rows=1 width=2)
-> XN Hash IN Join DS_DIST_NONE (cost=2409.40..4591.98 rows=2031 width=2)
Hash Cond: ("outer".listid = "inner".listid)
-> XN Seq Scan on sales (cost=0.00..1724.55 rows=172455 width=6)
-> XN Hash (cost=2406.20..2406.20 rows=1176 width=4)
-> XN Seq Scan on listing (cost=0.00..2406.20 rows=1176 width=4)
Filter: (listtime > '2008-12-26 00:00:00':timestamp without time zone)

At the bottom right, there is a status bar showing 'Query ID 410515', 'Elapsed time: 13 ms', and 'Total rows: 7'.

Run Limit 100 Explain Isolated session Serverless: de... sample_ticket...

Schedule

```

9  explain
10 select sum(qtysold)
11 from sales
12 where saletime > '2008-12-26'
13 and listid in (select listid from listing where listtime > '2008-12-26');
14

```

Row 8, Col 1, Chr 277

Result 1 (7) Result 2 (8) Export Chart

QUERY PLAN

- XN Aggregate (cost=4572.26..4572.26 rows=1 width=2)
 - > XN Hash IN Join DS_DIST_NONE (cost=2409.40..4568.79 rows=1387 width=2)
 - Hash Cond: ("outer".listid = "inner".listid)
 - > XN Seq Scan on sales (cost=0.00..2155.69 rows=1387 width=6)
 - Filter: (saletime > '2008-12-26 00:00:00'::timestamp without time zone)
 - > XN Hash (cost=2406.20..2406.20 rows=1176 width=4)
 - > XN Seq Scan on listing (cost=0.00..2406.20 rows=1176 width=4)
 - Filter: (listtime > '2008-12-26 00:00:00'::timestamp without time zone)

Query ID 410515 Elapsed time: 58 ms Total rows: 8

(2) A brief discussion comparing the two queries in terms of their expected output and query costs.

Both queries returned the same result (sum = 46) because every sale tied to listings posted after 2008-12-26 also happened after this date, so the extra saletime filter in the second query doesn't exclude anything. The second query is cheaper because it filters the sales by date first, so there is less data to compare. The first query filters later, so it has to scan more rows before joining.

```

5
6 select sum(qtysold)
7 from sales
8 where listid in (select listid from listing where listtime > '2008-12-26');
9

```

Row 2, Col 20, Chr 534

Result 1 (7) Result 2 (1) Result 3 (1) Result 4 (8) Export Chart

sum
46

```

10
11 select sum(qtysold)
12 from sales
13 where saletime > '2008-12-26'
14 and listid in (select listid from listing where listtime > '2008-12-26');

```

Row 2, Col 20, Chr 534

Result 1 (7) Result 2 (1) Result 3 (1) Result 4 (8) Export Chart

sum
46

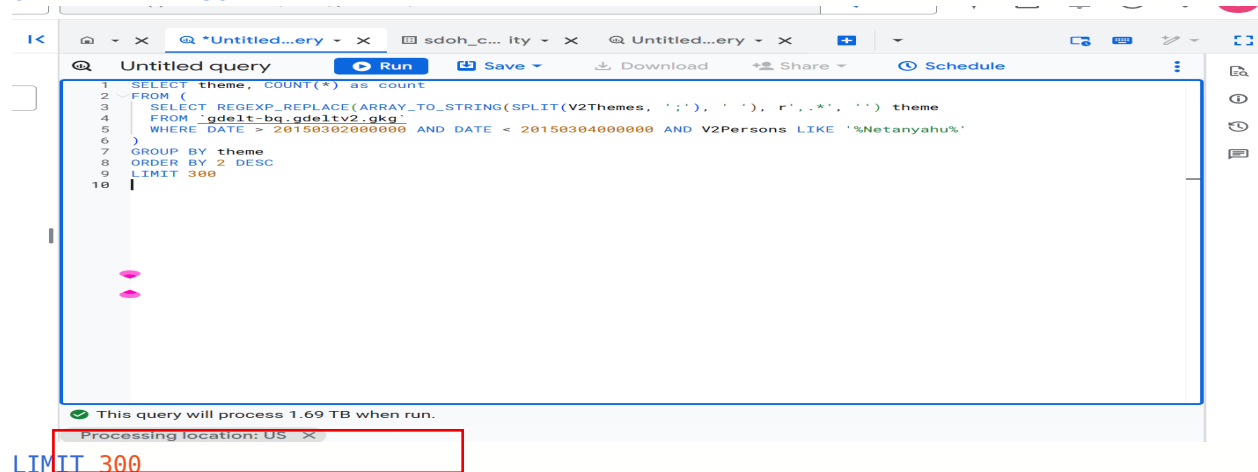
2. Google BigQuery

a. Load the sample data: GDELT, run the following query and calculate the execution cost:

Often you want to be able to run a query on the GKG and get back a list of the top people, organizations, general names, or themes that appear in matching coverage. For example, imagine creating a table of the top themes associated with Israeli Prime Minister Benjamin Netanyahu during his visit to the US Congress on March 3, 2015.

CAUTION: The query below may exhaust your monthly free limit on BigQuery! Optimize the query below to reduce query costs, and verify query costs before running the query.

```
SELECT theme, COUNT(*) as count
FROM (
  SELECT REGEXP_REPLACE(ARRAY_TO_STRING(SPLIT(V2Themes, ';'), ' '), r',.*', '') theme
  FROM `gdelt-bq.gdeltv2.gkg`
  WHERE DATE > 20150302000000 AND DATE < 20150304000000 AND V2Persons LIKE
  '%Netanyahu%'
)
GROUP BY theme
ORDER BY 2 DESC
LIMIT 300
```



On-Demand

Execution cost

Amount of data queried 1.7 TB



Location type Region

Location Iowa (us-central1)

The screenshot shows the Google Cloud BigQuery console. At the top, there's a toolbar with buttons for 'Run', 'Save', 'Download', 'Share', and 'Schedule'. Below the toolbar, the query editor contains the following SQL code:

```

1 SELECT theme, COUNT(*) as count
2 FROM (
3   SELECT REGEXP_REPLACE(ARRAY_TO_STRING(SPLIT(V2Themes, ','), ' '), r'\.+', '') theme
4   FROM `gdelt-bq.gdeltv2.gkg`
5   WHERE DATE > 20150302000000 AND DATE < 20150304000000 AND V2Persons LIKE '%Netanyahu%'
6 )
7 GROUP BY theme
8 ORDER BY 2 DESC
9 LIMIT 300
10

```

Below the query editor, a status bar indicates 'Query completed' and 'Processing location: US'. The 'Query results' section is active, showing a table with 5 rows and 2 columns: 'theme' and 'count'.

Row	theme	count
1	TAX_POLITICAL_PARTY_REPUB...	2088
2	LEADER	1217
3	MEDIA_MSM	989
4	TERROR	785
5	TAX_FNCACT_SENATORS	467

At the bottom right, there's a pagination control showing 'Results per page: 50' and '1 - 50 of 300'.

b. Examine a public dataset: *nativity*

Examine the *nativity* table schema in the samples dataset using *bq show* command, and Use the *bq query* command to run the following SQL query on data to extract How many children were born to mother's over 50 in the ten states with the most births:

```

SELECT
  mother_age,
  COUNT(mother_age) total
FROM
  `bigquery-public-data.samples.nativity`
WHERE
  state IN (SELECT
    state
  FROM
    (SELECT
      state,
      COUNT(state) total
    FROM
      `bigquery-public-data.samples.nativity`
    GROUP BY
      state
    ORDER BY
      total DESC
    LIMIT 20))
  AND mother_age > 50
GROUP BY
  mother_age
ORDER BY
  mother_age DESC;

```

Submit:

(1). Screenshots of the query result and cost.

The screenshot shows a BigQuery console window with an 'Untitled query' editor. The query is as follows:

```
1 SELECT
2   mother_age,
3   COUNT(mother_age) total
4 FROM
5   `bigquery-public-data.samples.natality`
6 WHERE
7   state IN (SELECT
8             state
9             FROM
10              (SELECT
11                state,
12                COUNT(state) total
13              FROM
14                `bigquery-public-data.samples.natality`
15              GROUP BY
16                state
17              ORDER BY
18                total DESC
19              LIMIT 10))
20 AND mother_age > 50
21 GROUP BY
22   mother_age
23 ORDER BY
24   mother_age DESC;
25
```

Below the query editor, a status bar indicates: ✓ This query will process 1.48 GB when run.

The screenshot shows the BigQuery cost calculator interface. It displays the pricing model as 'On-Demand' and the cost as '\$0.00'. A red box highlights the 'Amount of data queried' as '1.5 GB' and the cost as '\$0.00'. Below this, the 'Location type' is set to 'Region' and the 'Location' is set to 'Iowa (us-central1)'.

Query results				Save results	Open in	
Job information				Results	Visualization	JSON
Execution details				Execution graph		
Row	mother_age	total				
1	54	75				
2	53	112				
3	52	154				
4	51	276				

Results per page: 50 1 - 4 of 4

(2). Screenshots of command results from *bq show* and *bq query*.

'bq show' command result:

Table bigquery-public-data:samples.natality								
Last modified	Schema	Total Rows	Total Bytes	Expiration	Time Partitioning	Clustered Fields	Total Logical Bytes	
Total Physical Bytes	Labels							
14 Mar 17:16:47 1209802476	- source_year: integer (required) - year: integer - month: integer - day: integer - wday: integer - state: string - is_male: boolean (required) - child_race: integer - weight_pounds: float - plurality: integer - apgar_1min: integer - apgar_5min: integer - mother_residence_state: string - mother_race: integer	137826763	23562717384				23562717384	

```
| - mother_age: integer
| - gestation_weeks: integer
| - lmp: string
| - mother_married: boolean
| - mother_birth_state: string
| - cigarette_use: boolean
| - cigarettes_per_day: integer
| - alcohol_use: boolean
| - drinks_per_week: integer
| - weight_gain_pounds: integer
| - born_alive_alive: integer
| - born_alive_dead: integer
| - born_dead: integer
| - ever_born: integer
| - father_race: integer
| - father_age: integer
| - record_weight: integer
```

'bq query' command result:

```
+-----+-----+
| mother_age | total |
+-----+-----+
|          54 |    75 |
|          53 |   112 |
|          52 |   154 |
|          51 |   276 |
+-----+-----+
```